

# Lecture 9: Introduction to Generative Models



Just kidding- They don't exist

# Today:

**00:00)** Goal, motivation, basic methods

**00:10)** VAE

**00:25)** GANs

**00:45)** ---- Break ----

**01:00)** GANs

**01:20)** Evaluation of generative models

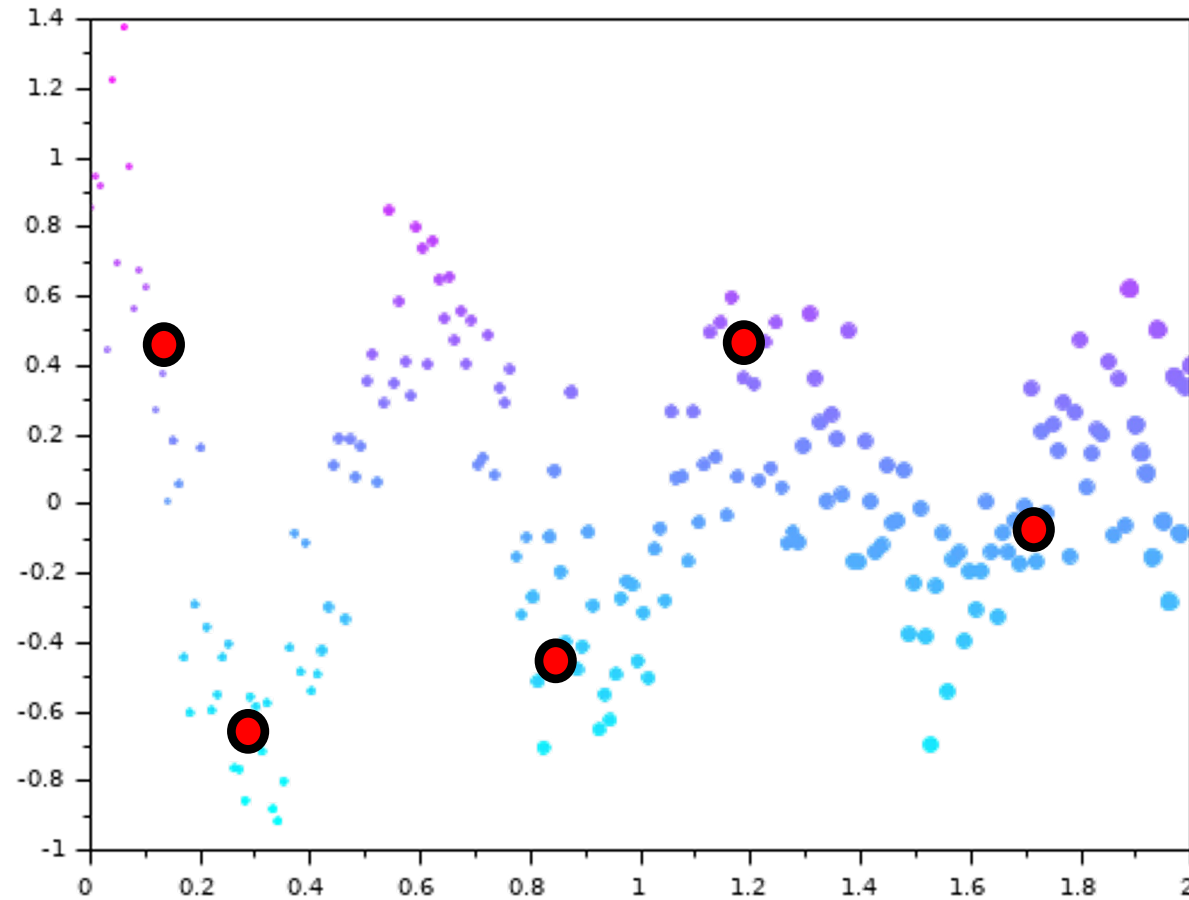
**01:45)** ---- Break ----

**02:00)** Intro to diffusion models

**02:45)** ---- Break ----

**03:00)** Tutorial: CLIP, Latent Diffusion & DiT

# Generative models: Not as sexy as you thought



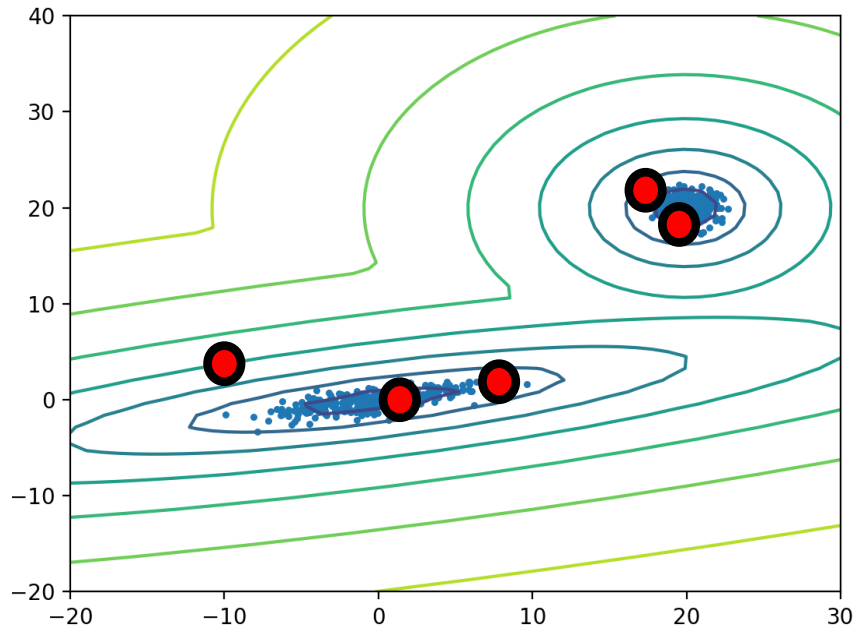
# Generative methods:

- Parametric distribution estimation (e.g. GMM)
- Autoregressive models (e.g. RNN, Causal CNN, Diffusion)
- Latent space mapping (e.g. VAE, GAN)

# Parametric Distribution Estimation

## Example: GMM

**Step 1: observe a set of samples**



**Step 2: assume a GMM model**

$$p(x|\theta) = \sum_i \pi_i \mathcal{N}(x|\mu_i, \Sigma_i)$$

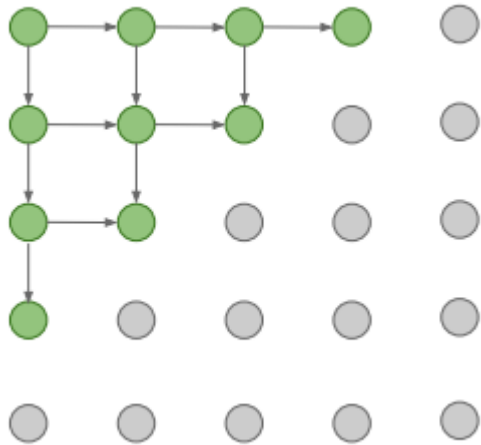
**Step 3: perform maximum likelihood learning**

$$\max_{\theta} \sum_{x^{(j)} \in \text{Dataset}} \log p(\theta|x^{(j)})$$

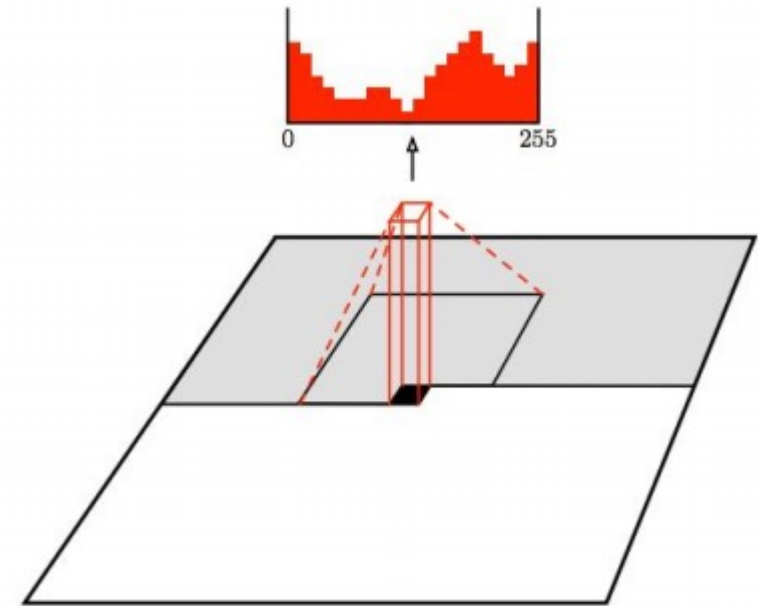
**Step 4: Sample**

# Autoregressive image generation

## PixelRNN



## PixelCNN

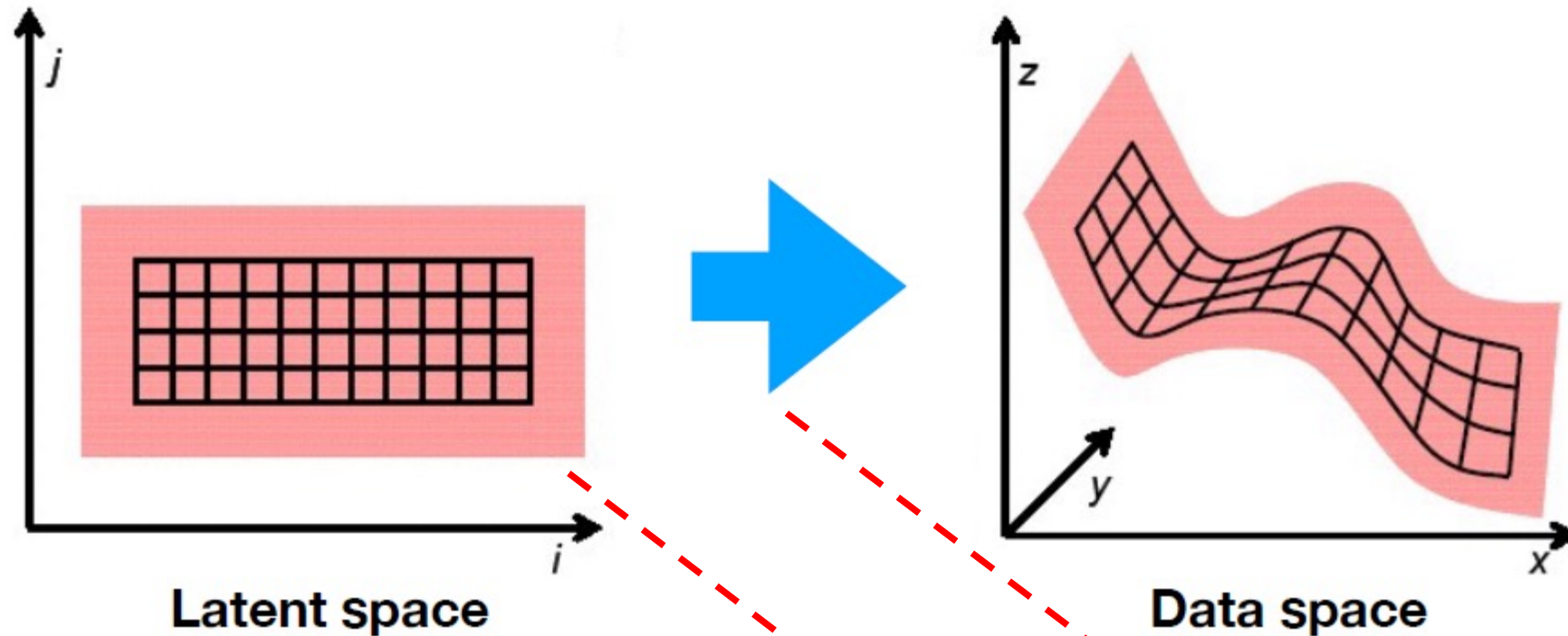


Van der Oord 2016

# Generative methods:

- Parametric distribution estimation (e.g. GMM)
- Autoregressive models (e.g. RNN, Causal CNN, Diffusion)
- Latent space mapping (e.g. VAE, GAN)

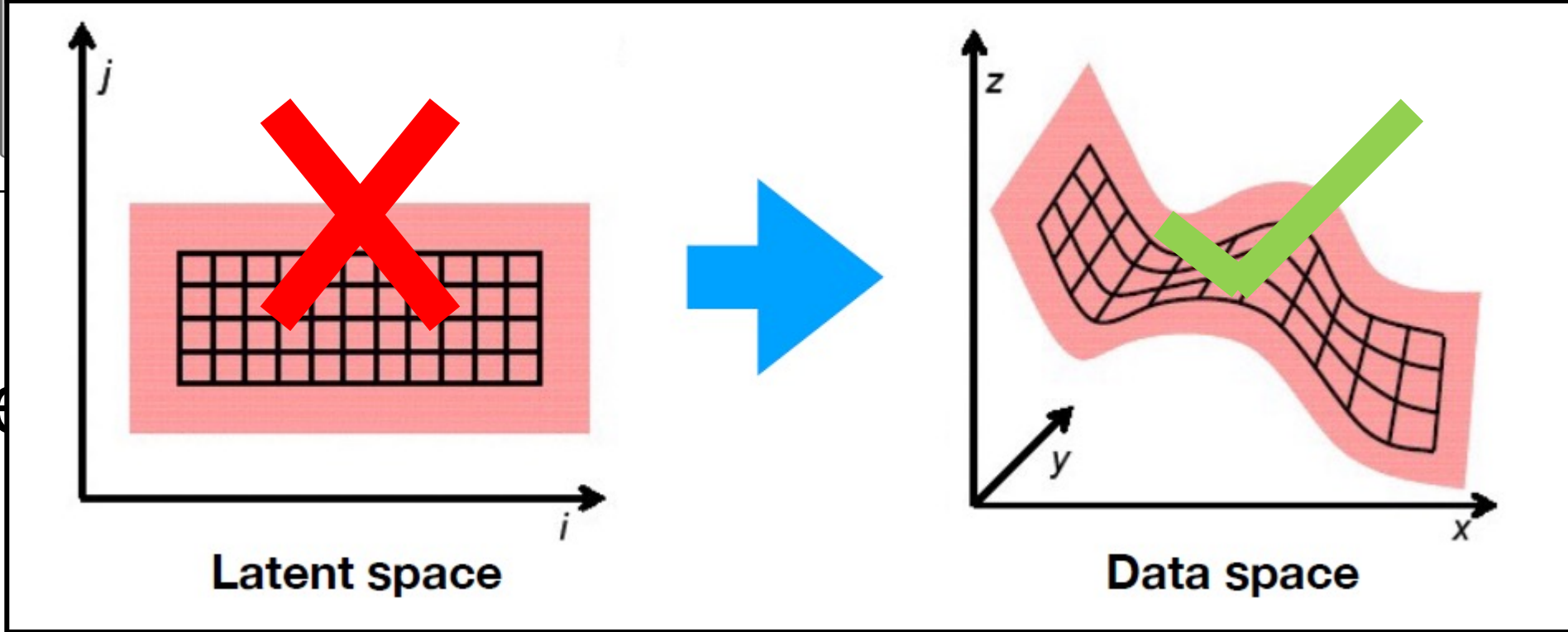
# Latent space mapping approach



Data likelihood: 
$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

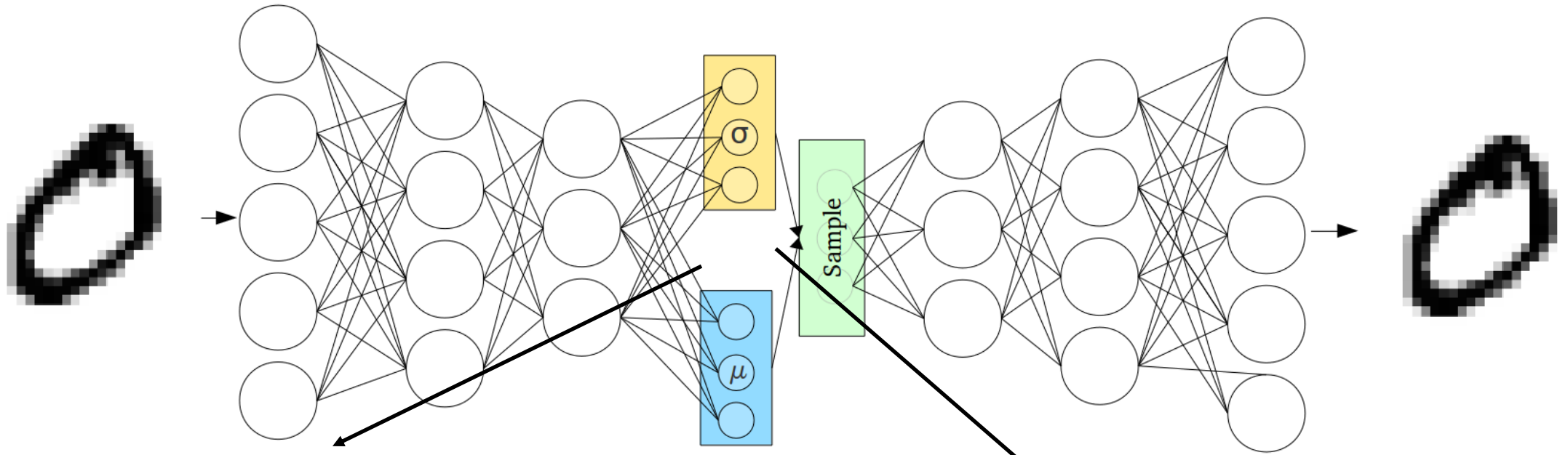
# Autoencoders

AE does not transform one *\*pre-determined\** distribution to another!



- Gene

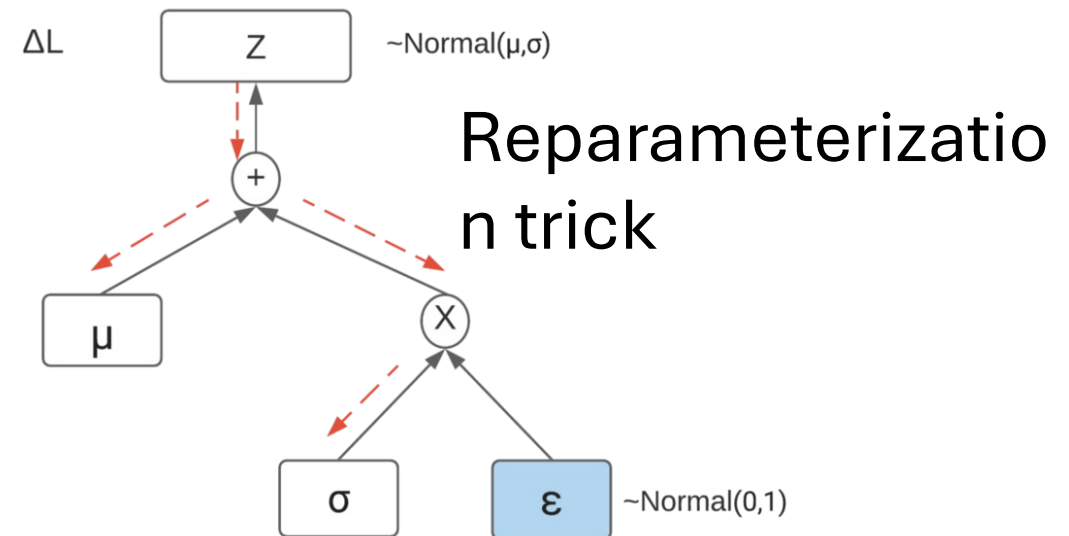
# Variational Autoencoders (Kingma&Welling 2014)



Regularization:  
encourage  $p(z) \sim \mathcal{N}(0,1)$

by KL divergence:

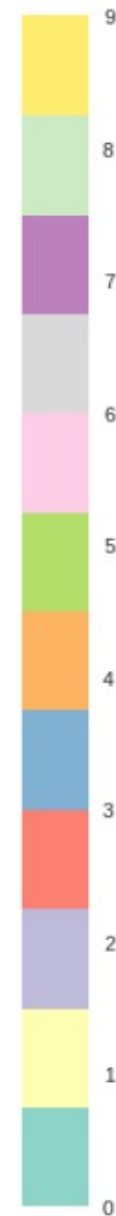
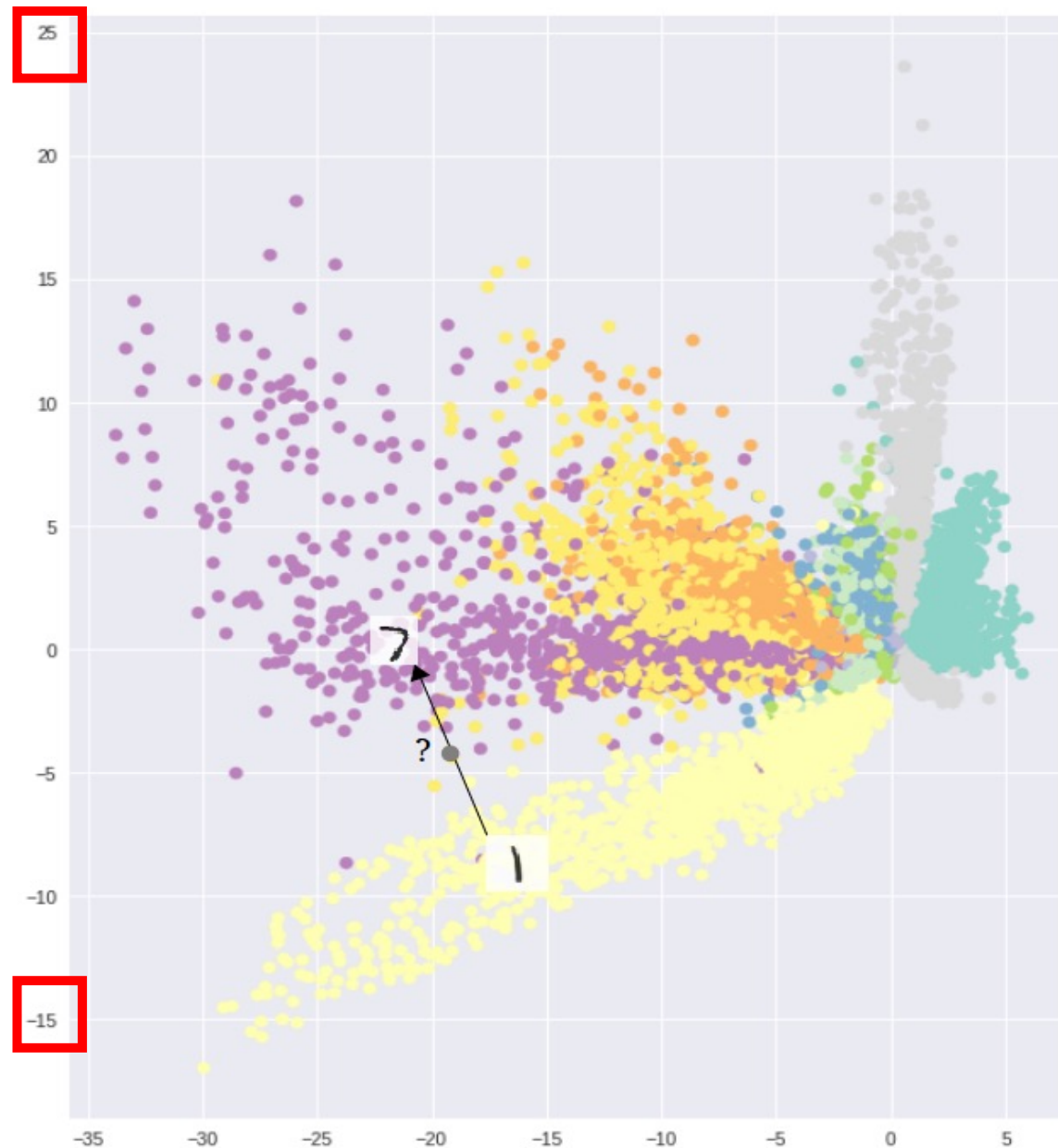
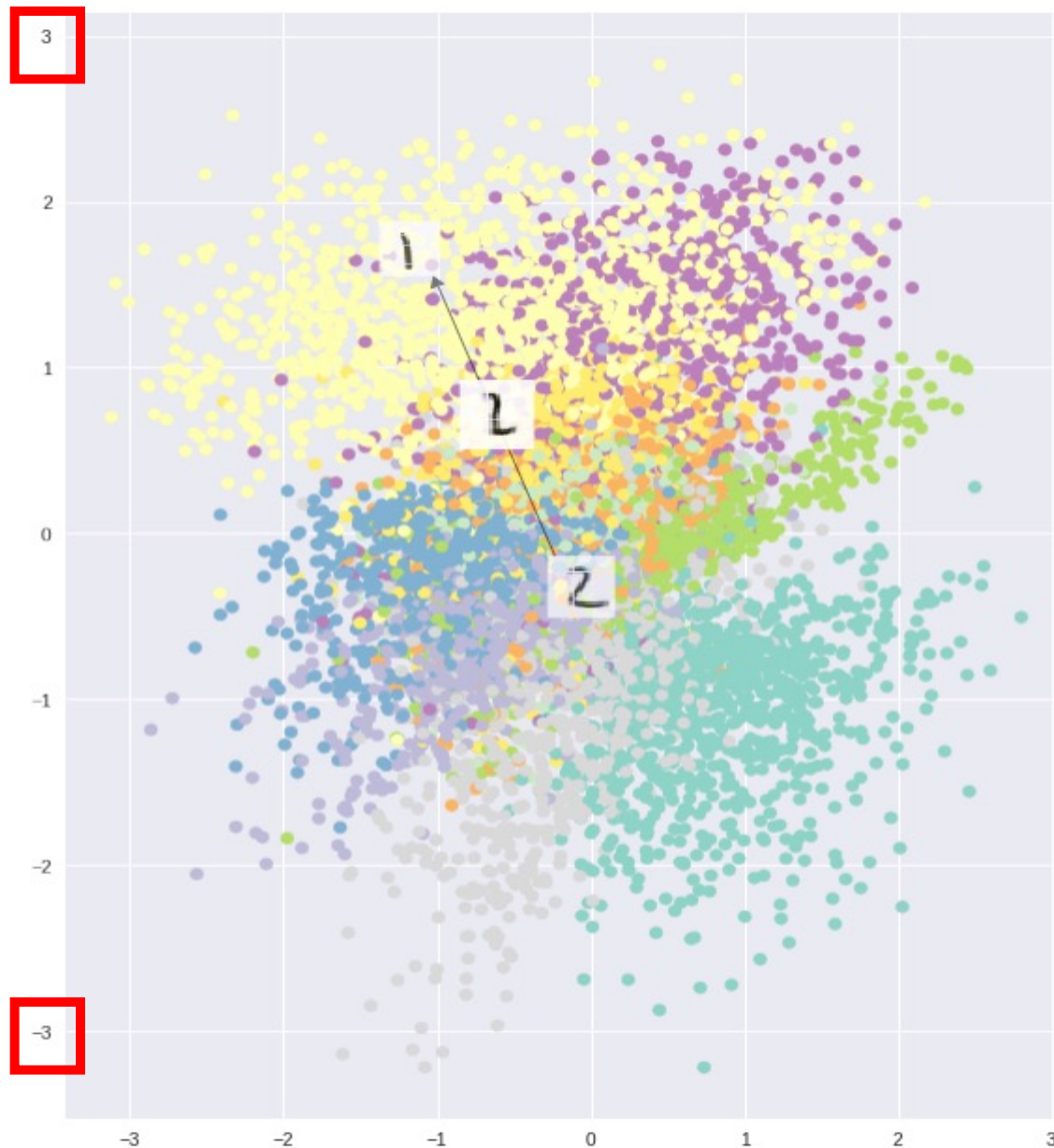
$$\sum_{i=1}^n \sigma_i^2 + \mu_i^2 - \log(\sigma_i) - 1$$



# VAE

Also check out the scale!

# AE

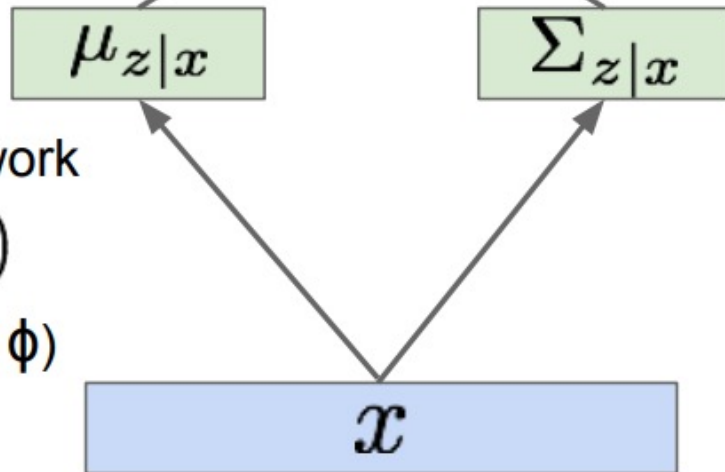


# Probabilistic interpretation

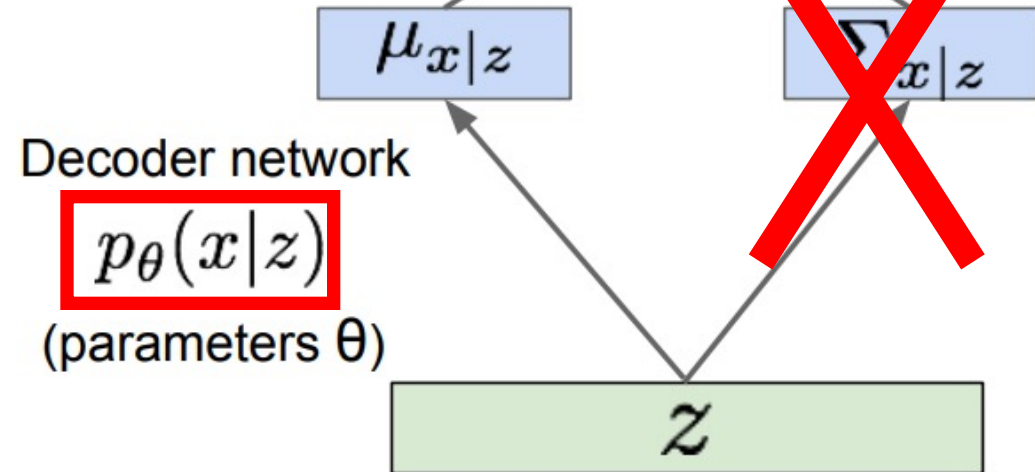
Data likelihood:  $p_{\theta}(x) = \int p_{\theta}(z) \overset{N(0,1)}{p_{\theta}(x|z)} dz$

Goal: make  $\log p_{\theta}(x^{(i)})$  high as possible

Sample  $z$  from  $z|x \sim \mathcal{N}(\mu_{z|x}, \Sigma_{z|x})$



Sample  $x|z$  from  $x|z \sim \mathcal{N}(\mu_{x|z}, \Sigma_{x|z})$



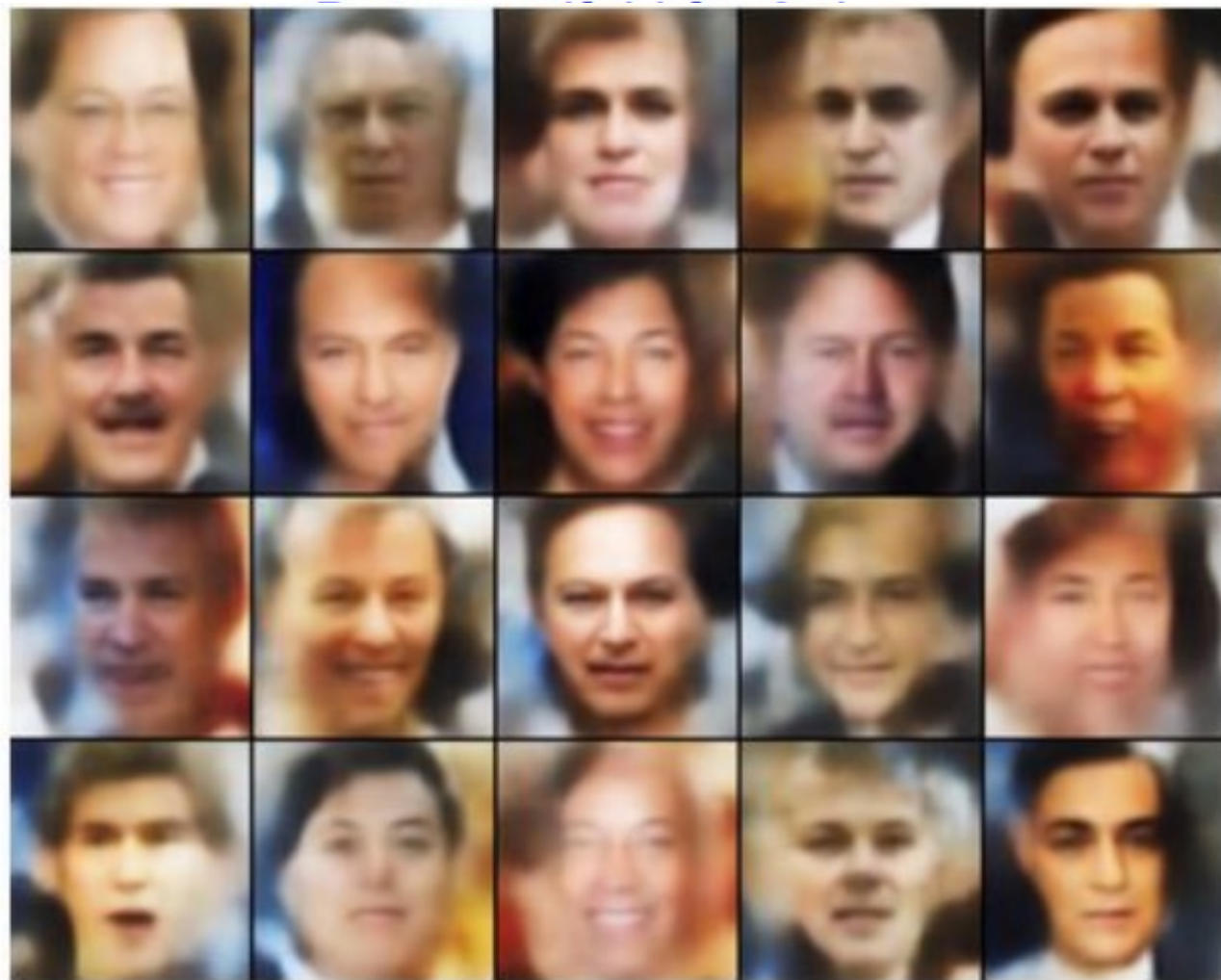
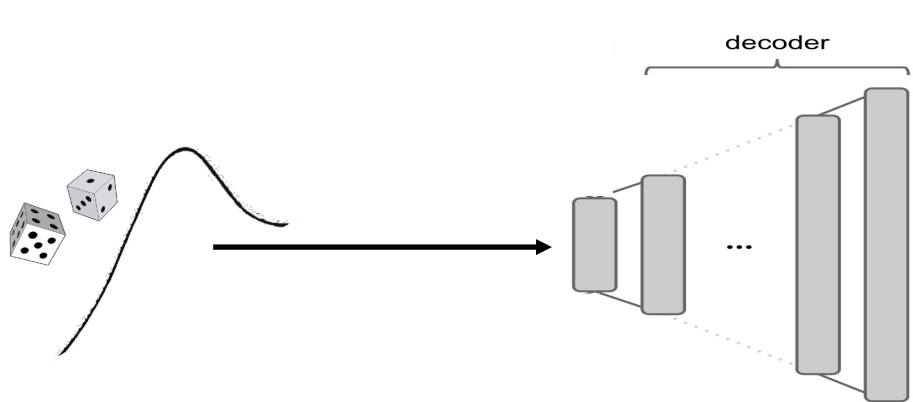
$$\begin{aligned}
\log p_{\theta}(x^{(i)}) &= \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} \left[ \log p_{\theta}(x^{(i)}) \right] && (p_{\theta}(x^{(i)}) \text{ Does not depend on } z) \\
&= \mathbf{E}_z \left[ \log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \right] && (\text{Bayes' Rule}) \\
&= \mathbf{E}_z \left[ \log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \frac{q_{\phi}(z | x^{(i)})}{q_{\phi}(z | x^{(i)})} \right] && (\text{Multiply by constant}) \\
&= \mathbf{E}_z \left[ \log p_{\theta}(x^{(i)} | z) \right] - \mathbf{E}_z \left[ \log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[ \log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z | x^{(i)})} \right] && (\text{Logarithms}) \\
&= \mathbf{E}_z \left[ \log p_{\theta}(x^{(i)} | z) \right] - D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z)) + D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z | x^{(i)}))
\end{aligned}$$

↑  
Decoder network gives  $p_{\theta}(x|z)$ , can compute estimate of this term through sampling. (Sampling differentiable through reparam. trick, see paper.)

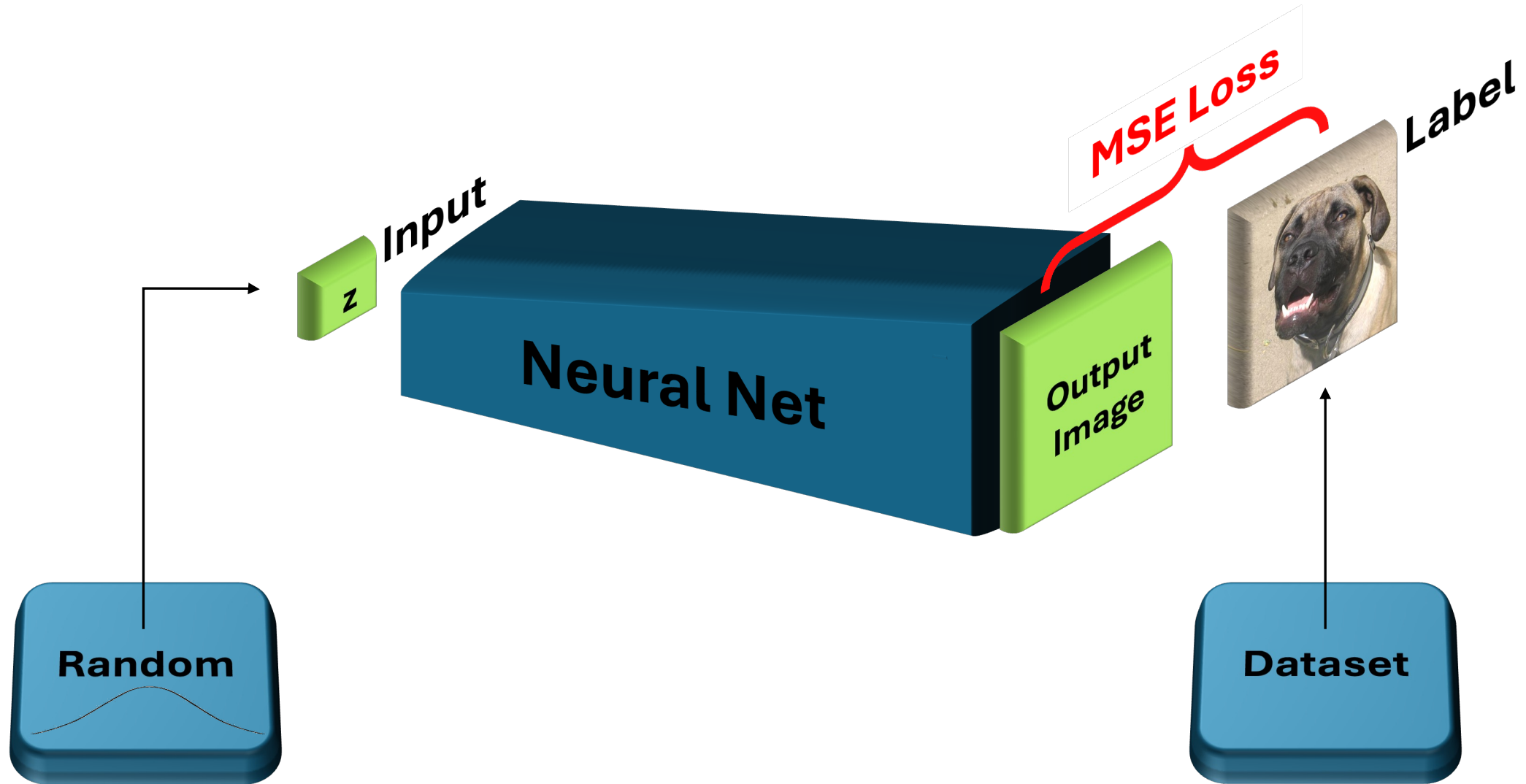
↑  
This KL term (between Gaussians for encoder and  $z$  prior) has nice closed-form solution!

↑  
 $p_{\theta}(z|x)$  intractable (saw earlier), can't compute this KL term :( But we know KL divergence always  $\geq 0$ .

# Generate data



# How about this idea for a generative model?

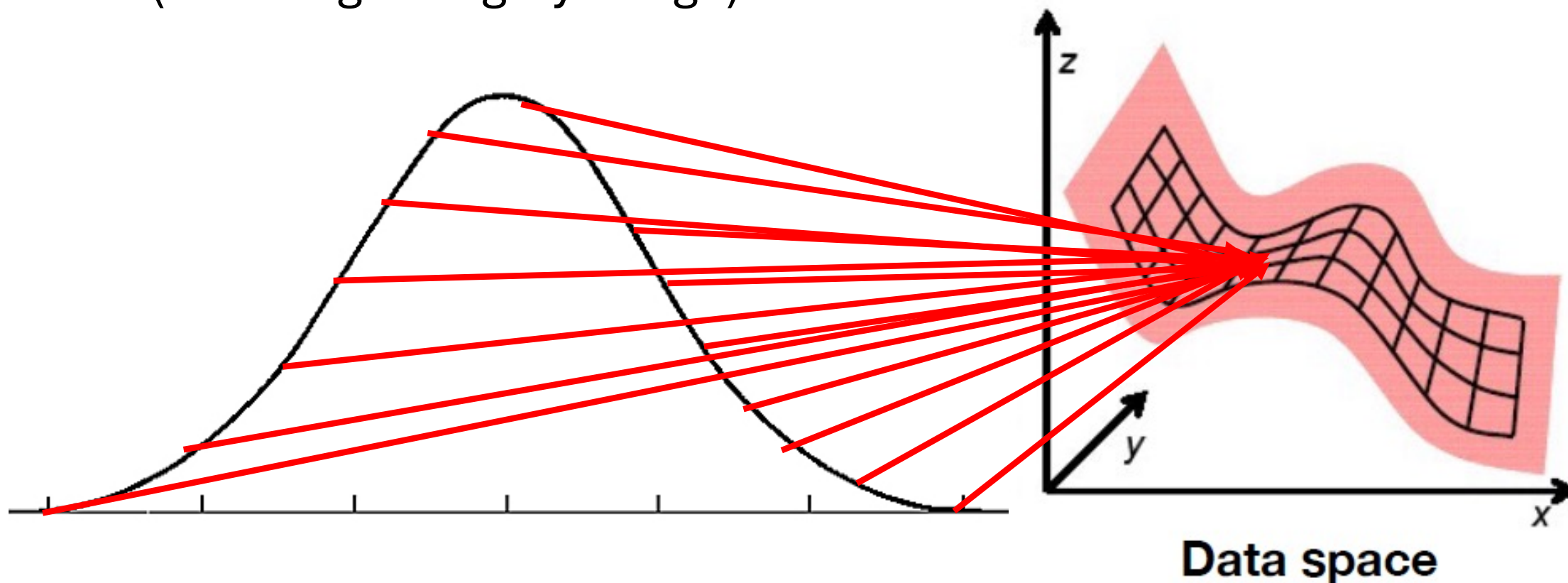


No good!

**Multimodality not obtained!**

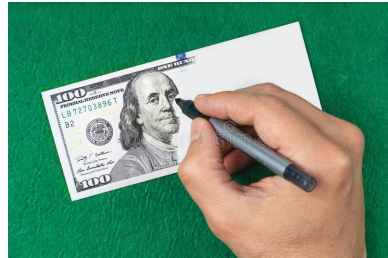
In expectation: every noise is mapped to every instance

Best L2 solution: All noise is mapped to the mean  
(For images:  $\sim$  grey image)



# Q: What makes a good counterfeiter?

## A: Can fool a good cop



*Fake (0) or  
Real (1) ?  
 $0 \leq p \leq 1$*

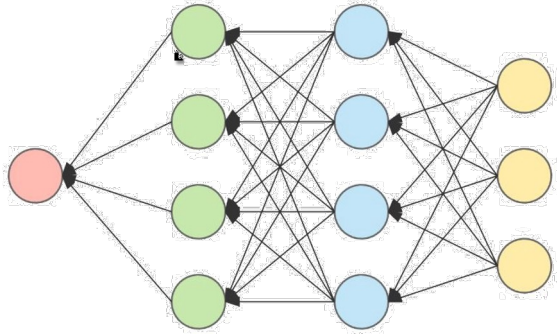


$$\min_G \max_D \left\{ \mathbb{E}_{x \sim p_{data}} \log(D(x)) + \mathbb{E}_{z \sim p_z} \log(1 - D(\mathbf{G}(z))) \right\}$$

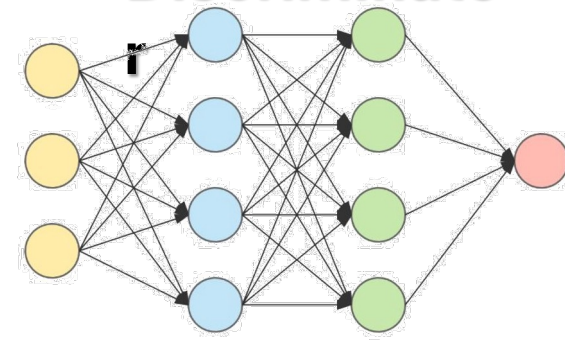
# Train D



**Generato**



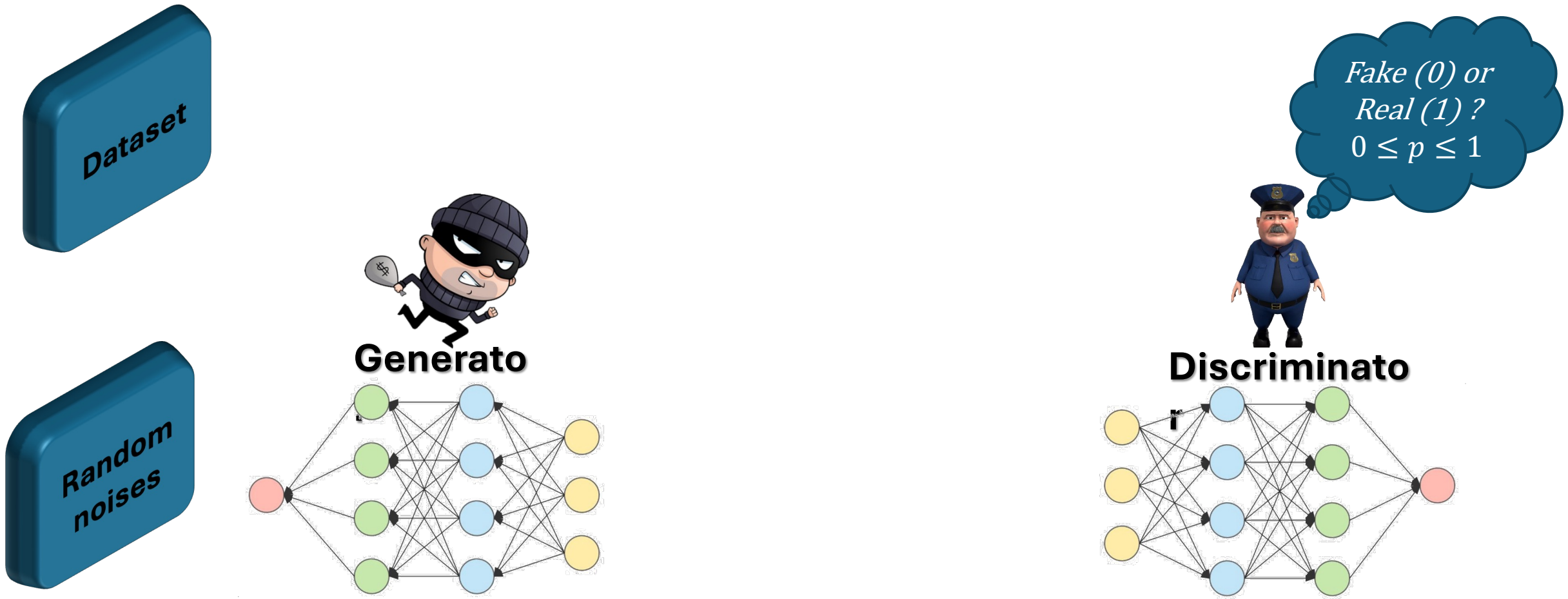
**Discriminato**



*Fake (0) or  
Real (1) ?  
 $0 \leq p \leq 1$*

$$\text{Maximize } \min_G \max_D \left\{ \mathbb{E}_{x \sim p_{data}} \log(D(x)) + \mathbb{E}_{z \sim p_z} \log(1 - D(\mathbf{G}(z))) \right\}$$

# Train G



$$\min_G \max_D \left\{ \mathbb{E}_{x \sim p_{data}} \log(D(x)) + \mathbb{E}_{z \sim p_z} \log(1 - D(\mathbf{G}(z))) \right\}$$

Data

Q: Who do you train first?

Random  
noise

A: Alternate training! G,D,G,D....

**Don't**

$$\min_G \max_D \left\{ \mathbb{E}_{x \sim p_{data}} \log(D(x)) + \mathbb{E}_{z \sim p_z} \log(1 - D(\mathbf{G}(z))) \right\}$$

# FAQ1: Why does it work?

- D learns probability! G trains to sample instance with high probability!
- Objective does not determine mapping directly- arrangement of latent space is learned!
- Theory: minimizes JS divergence between generated and real distributions.

# FAQ2: Why alternating?

- Gradients are meaningless when game is unbalanced.
- Pre-train D? Negative examples?
- Pre-train G? What loss?  
For G, D is a **learned loss function**



# GANs, Goodfellow 2014



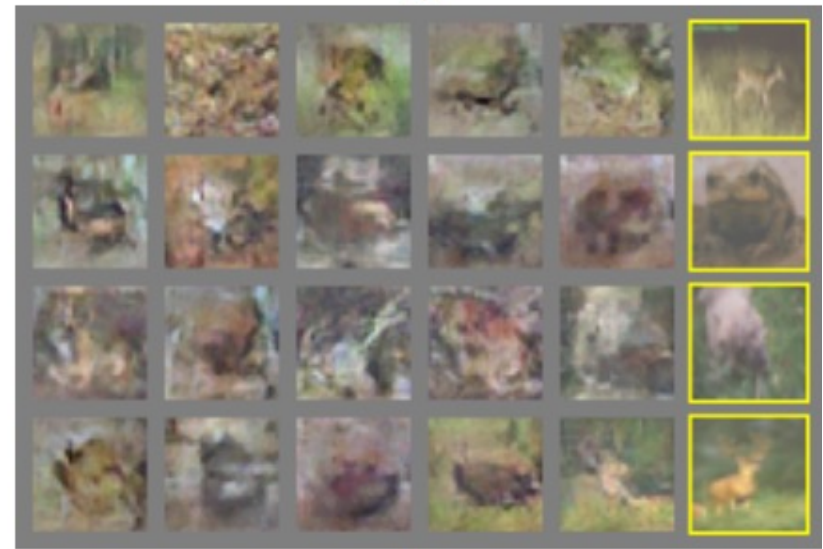
a)



b)



c)

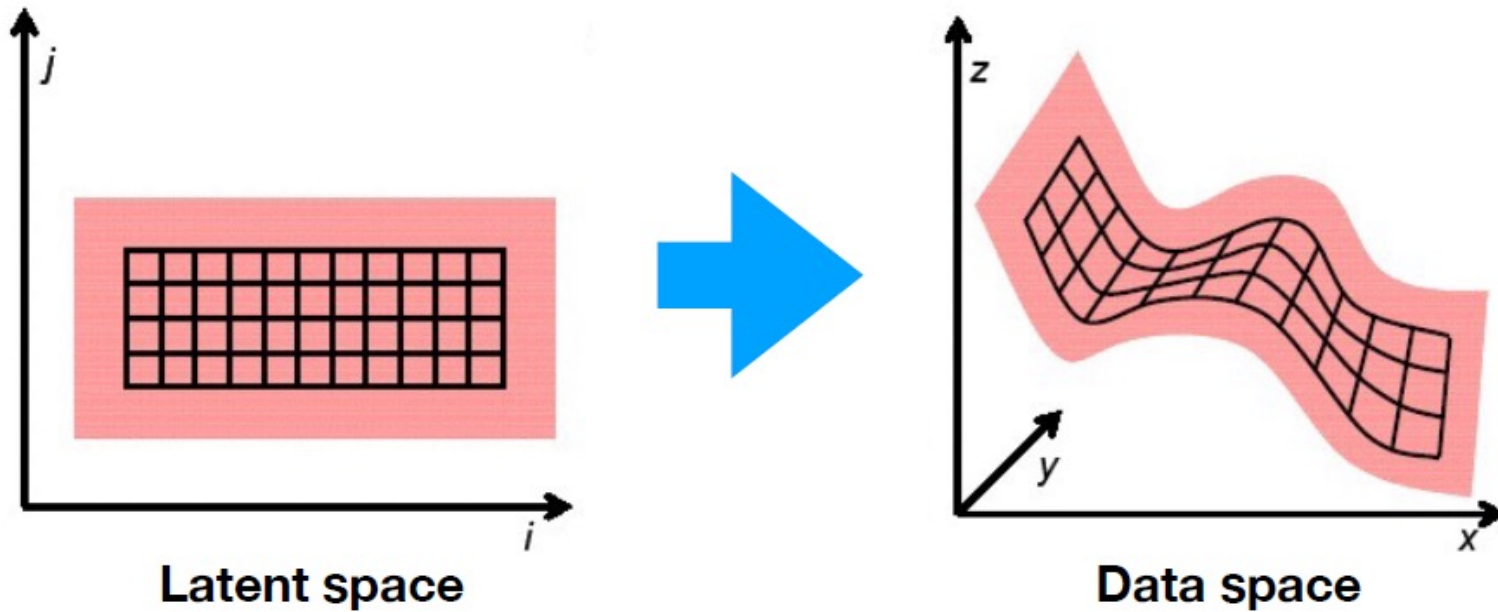


d)

# Latent space interpolation

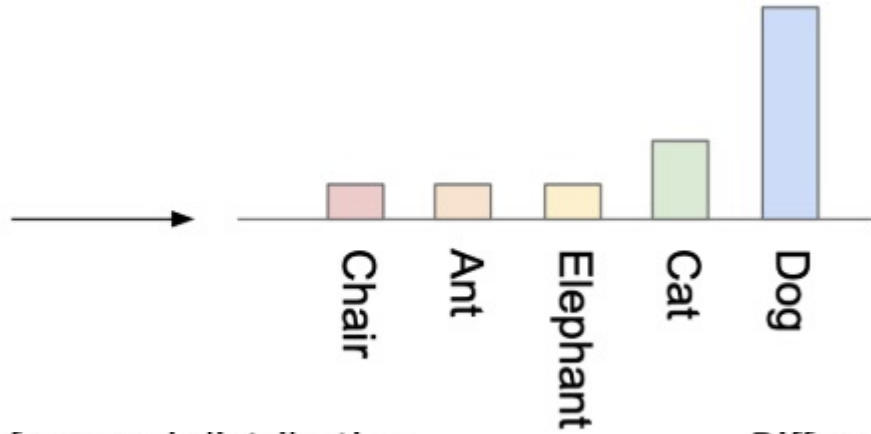


# Why does it work?



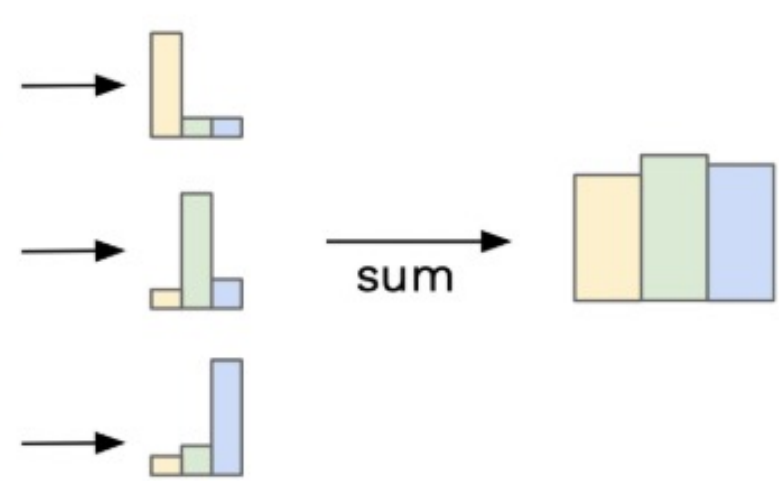
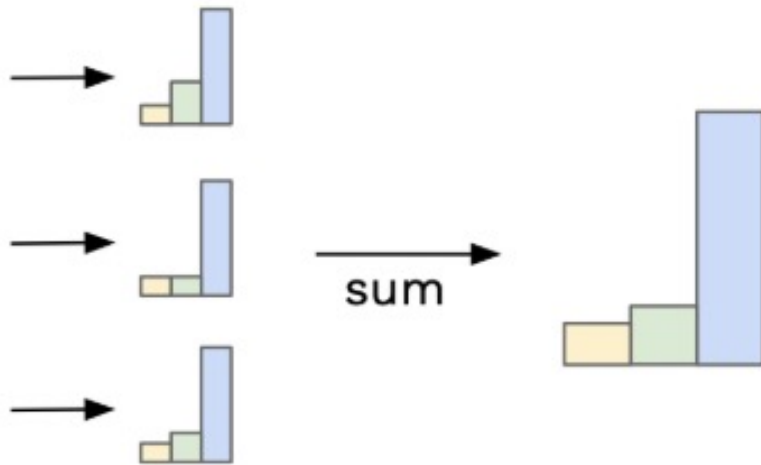
1. Every point is mapped to a valid example.
2. Network is continuous.

# Evaluation metrics: Inception score



Similar labels sum to give focussed distribution

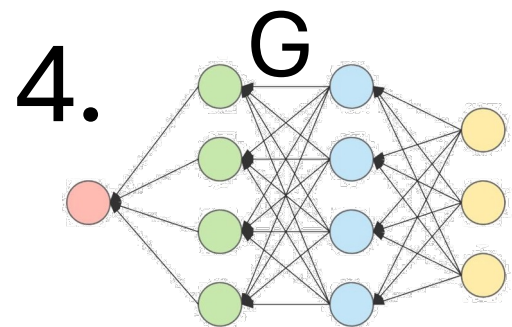
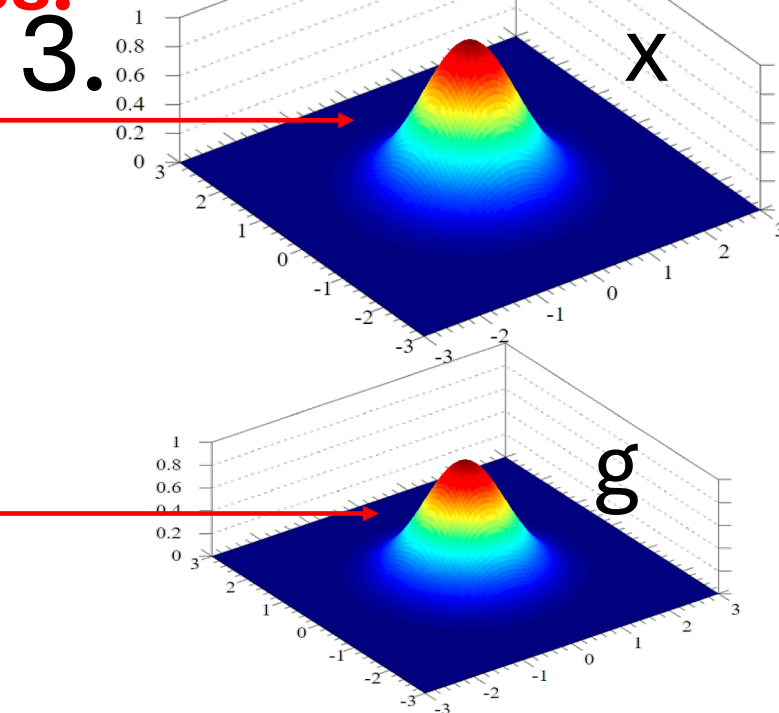
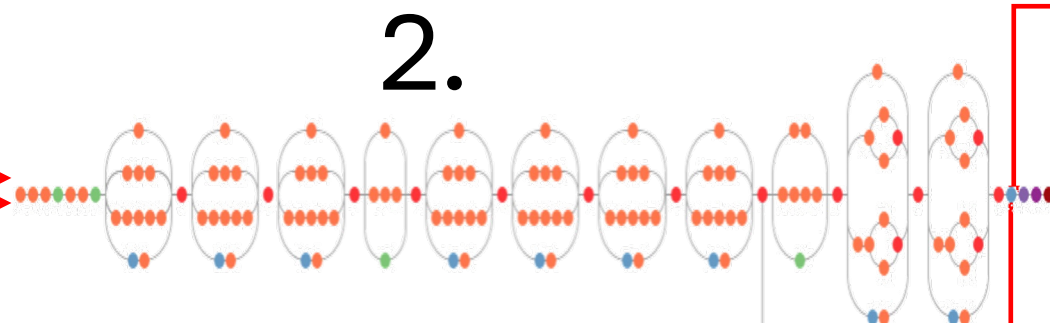
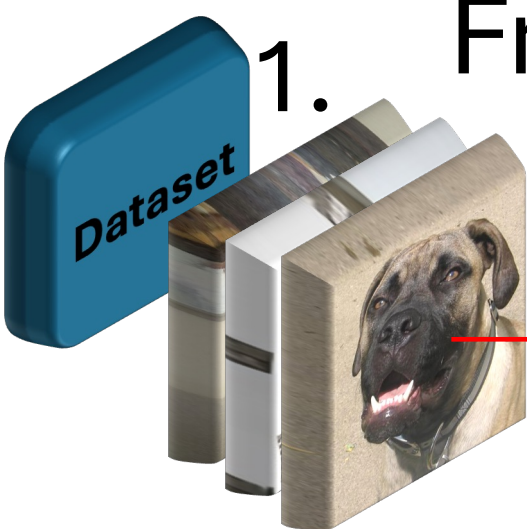
Different labels sum to give uniform distribution



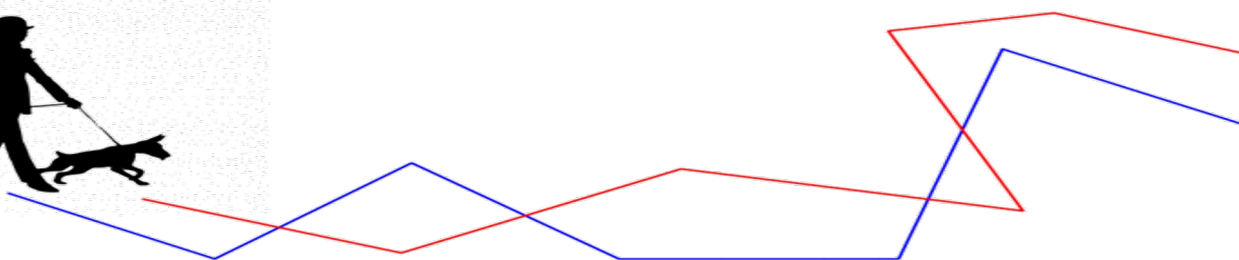
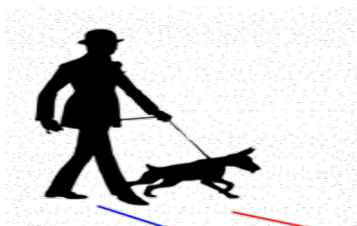
$$\text{IS}(G) = \exp \left( \mathbb{E}_{\mathbf{x} \sim p_g} D_{KL} \left( p(y|\mathbf{x}) \parallel p(y) \right) \right)$$

# 1. Fréchet Inception Distance (FID)

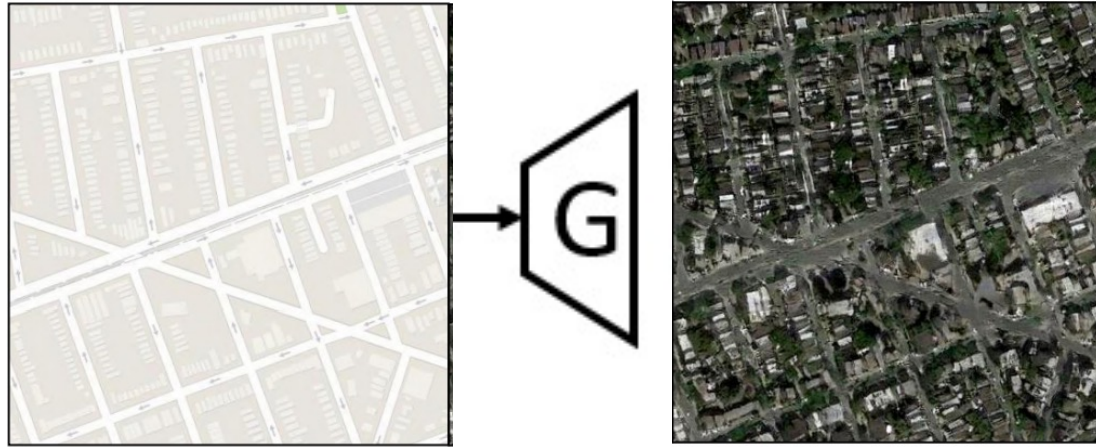
**Depends on the number of samples!**



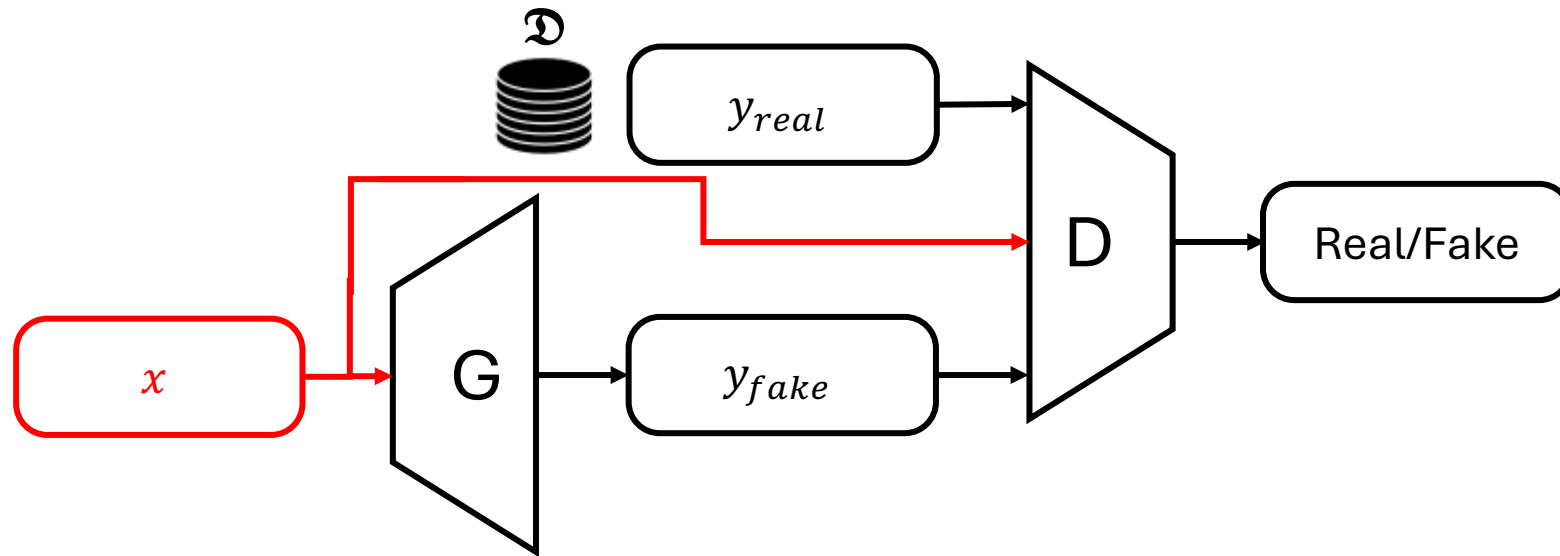
$$5. \text{FID}(x, g) = \|\mu_x - \mu_g\|_2^2 + \text{Tr}(\Sigma_x + \Sigma_g - 2(\Sigma_x \Sigma_g)^{\frac{1}{2}})$$



# Image to Image translation

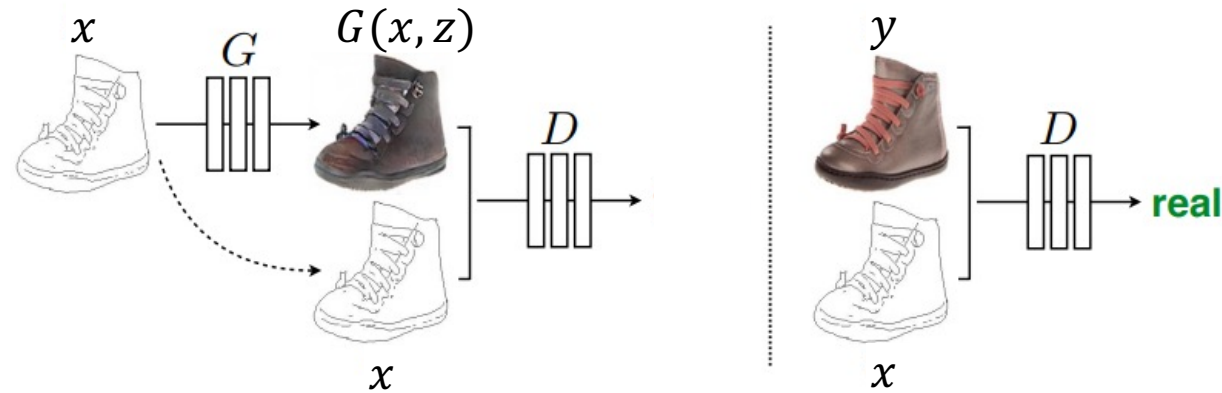


# Conditional GAN



$$\mathcal{L}_{\text{C-GAN}} = \min_G \max_D \mathbb{E}[\log D(y, \textcolor{red}{x})] + \mathbb{E}[\log(1 - D(G(\textcolor{red}{x}), \textcolor{red}{x}))]$$

# Pix2Pix



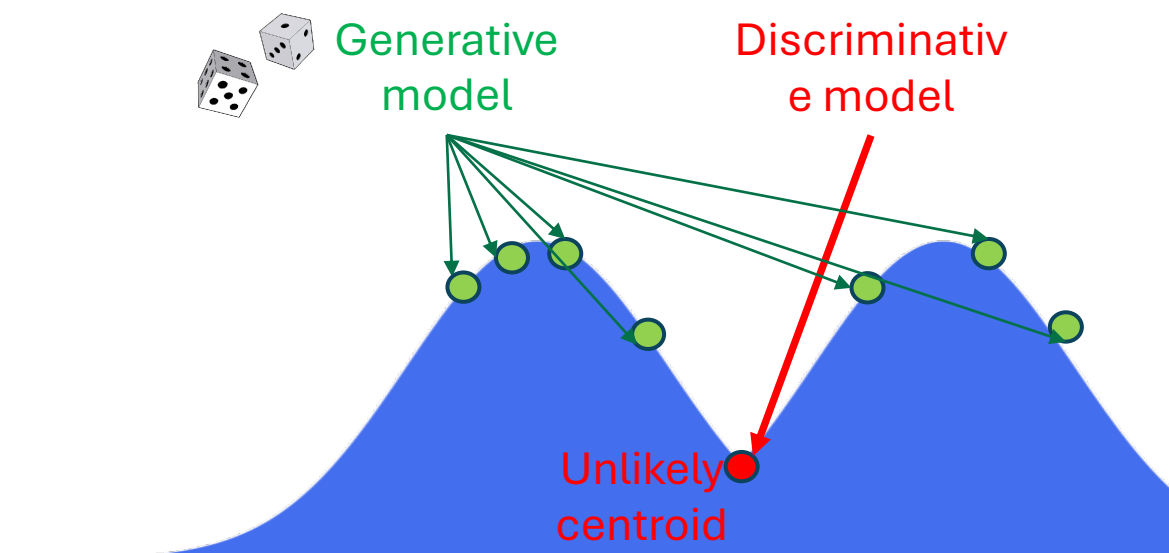
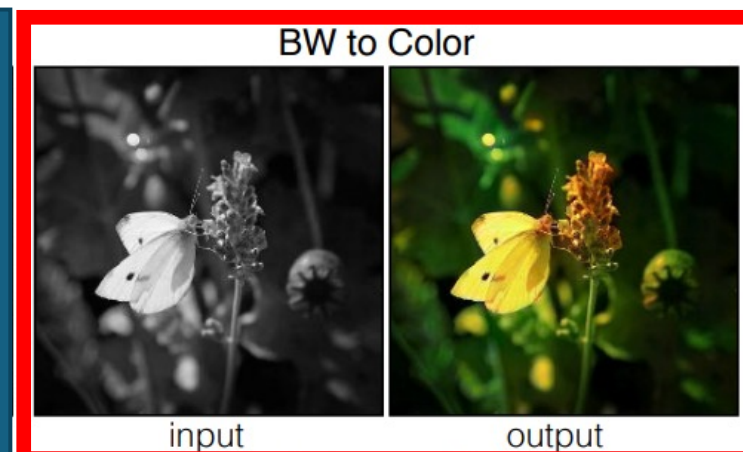
$$\mathcal{L}_{C-GAN} = \min_G \max_D \mathbb{E}[\log D(y, \boldsymbol{x})] + \mathbb{E}[\log(1 - D(G(\boldsymbol{x}), \boldsymbol{x}))]$$

$$\mathcal{L}_{L1} = \|y - G(x, z)\|_1$$

$$Objective = \mathcal{L}_{C-GAN} + \lambda \cdot \mathcal{L}_{L1}$$

# Generative VS Discriminative

What would happen if we train regular supervised mapping?



# Scaled up

Source A: gender, age, hair length, glasses, pose



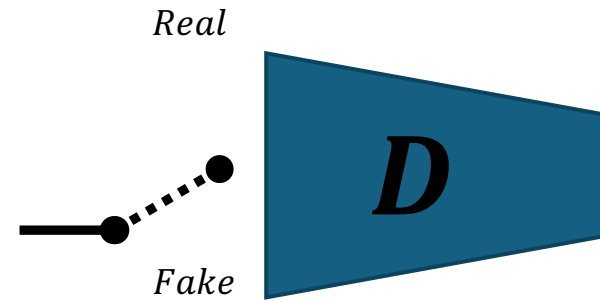
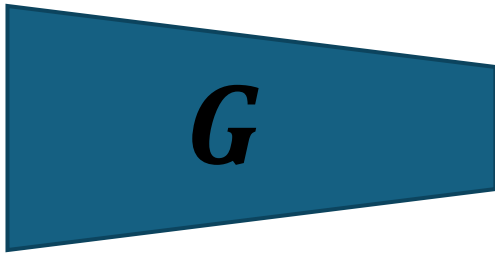
Source B:  
everything  
else



Result of combining A and B

# Training a GAN on a single image

*Input/noise*



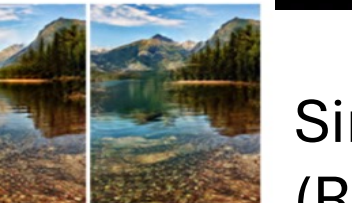
# Training a GAN on a single image

InGAN (Shocher,  
Bagon, Isola,  
Irani)



Input image

*Single training image*

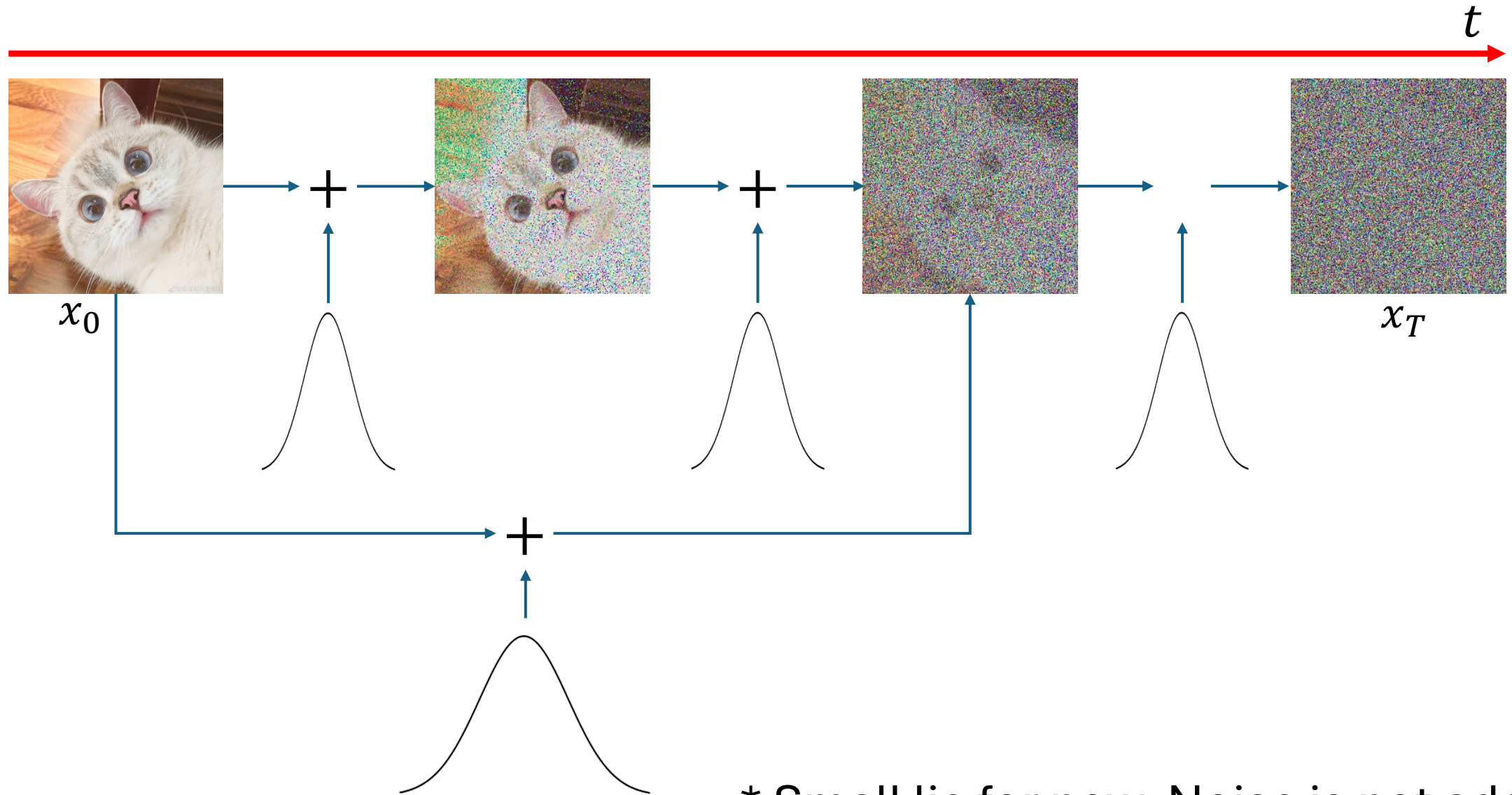


Random samples from a *single* image



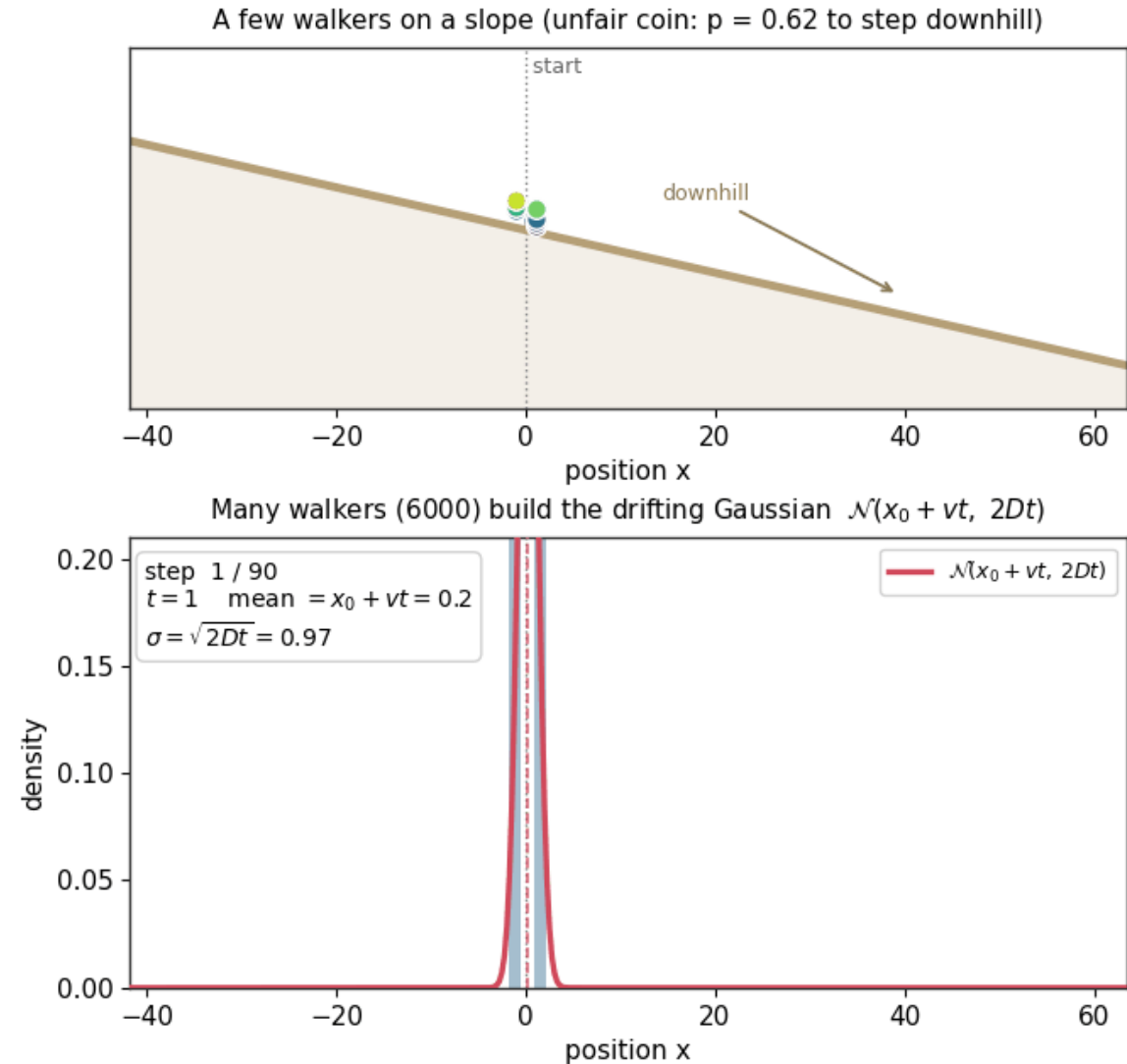
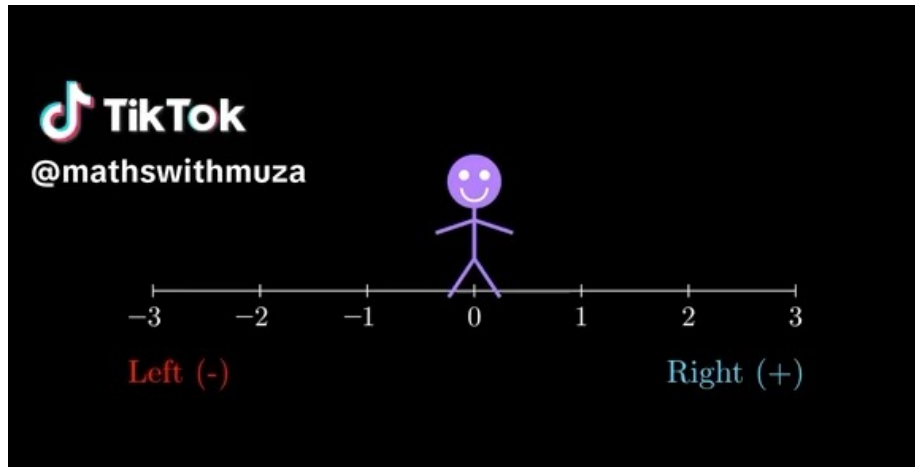
SinGAN  
(Rott-Shaham,  
Dekel,  
Michaelli)

# Diffusion models

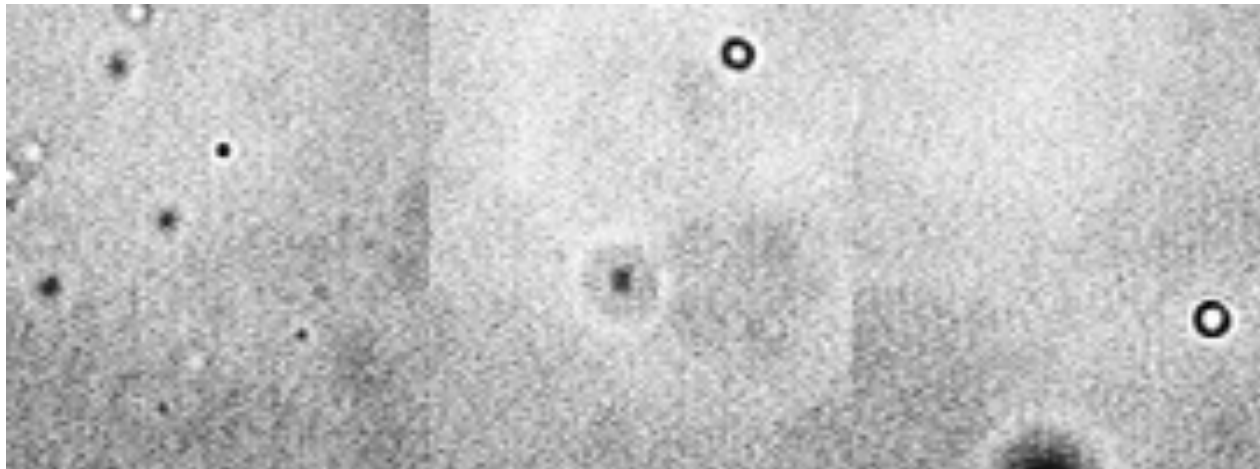
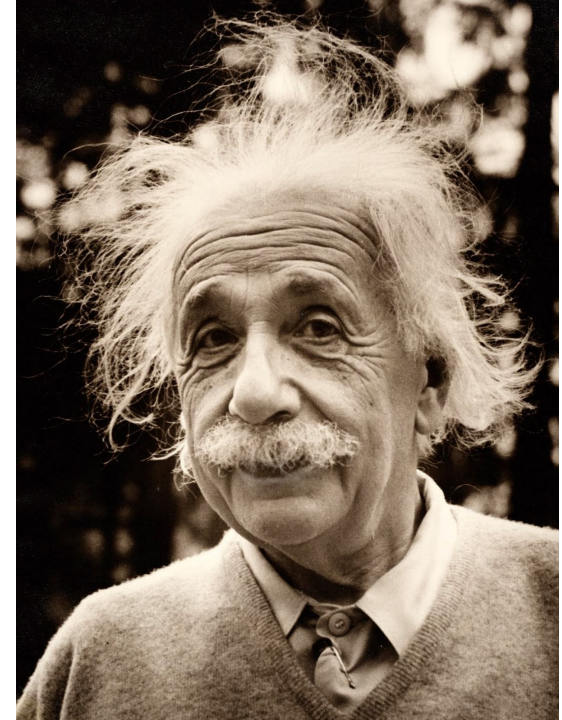
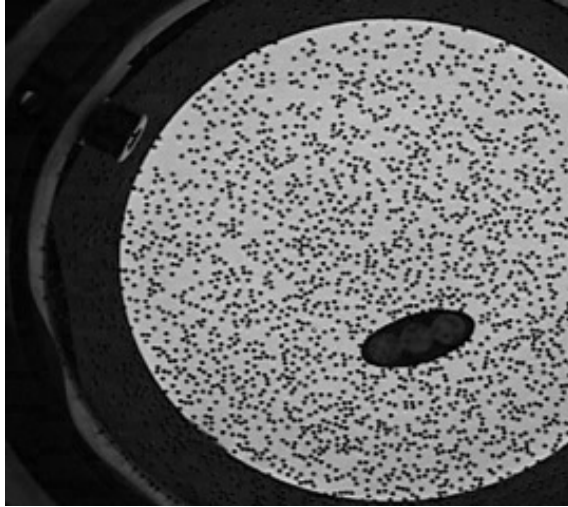
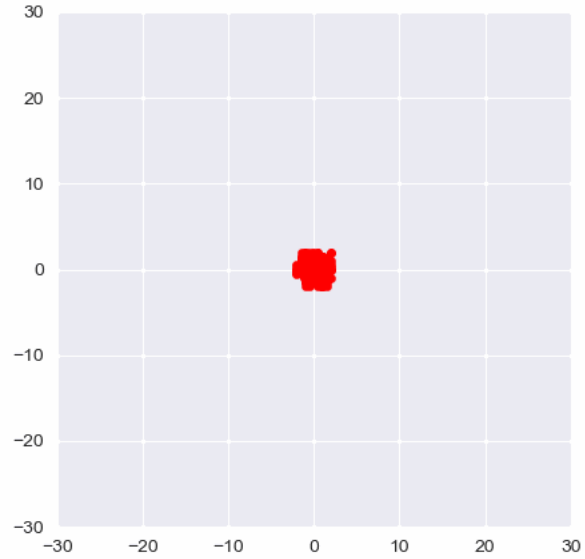


\* Small lie for now. Noise is not additive.

# Why is it called "Diffusion"?

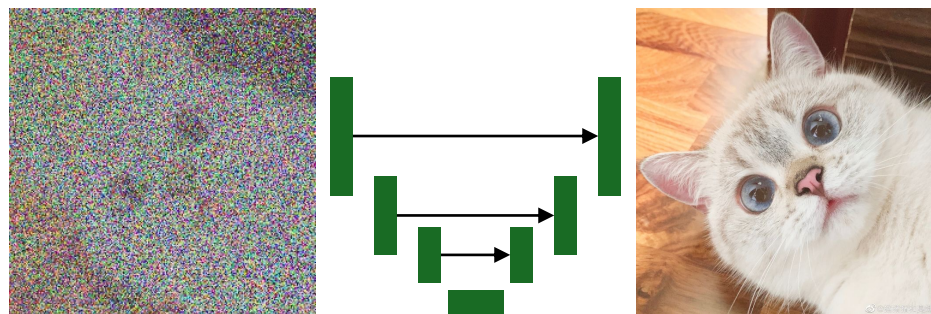


# Brownian motion



# Backward Process

Learn to denoise!

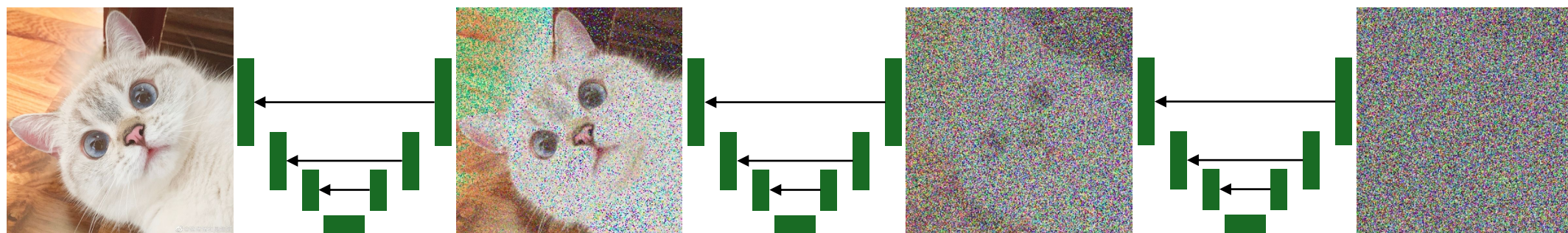


Use various noise levels.  
No need to noise sequentially.

$x_0$

Generate by autoregressive steps

$x_T$



Reverse Process

# Noise Schedule

$x_0$ : Data  
 $x_1/x_T$ : Source distrib sample  
 $\epsilon$ : Noise

| Framework                        | Forward interpolation                                    | Comments / coefficients                                           |
|----------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| Diffusion (DDPM / DDIM)          | $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$ | $\alpha_0 = 1$ , $\alpha_T = 0$                                   |
| EDM – Elucidated Diffusion Model | $x_t = x_0 + \sigma_t\epsilon$                           | “Exploding var”: $\sigma_{\min} \rightarrow \sigma_{\max}$        |
| Flow Matching (FM)               | $x_t = (1 - t)x_0 + tx_1$                                | Linear path, continuous<br>$0 < t < 1$                            |
| Cold Diffusion                   | $x_t = \Phi_t(x_0, \epsilon)$                            | Arbitrary user-defined degradation (blur, mask, downsample, etc.) |

# Predict what?!

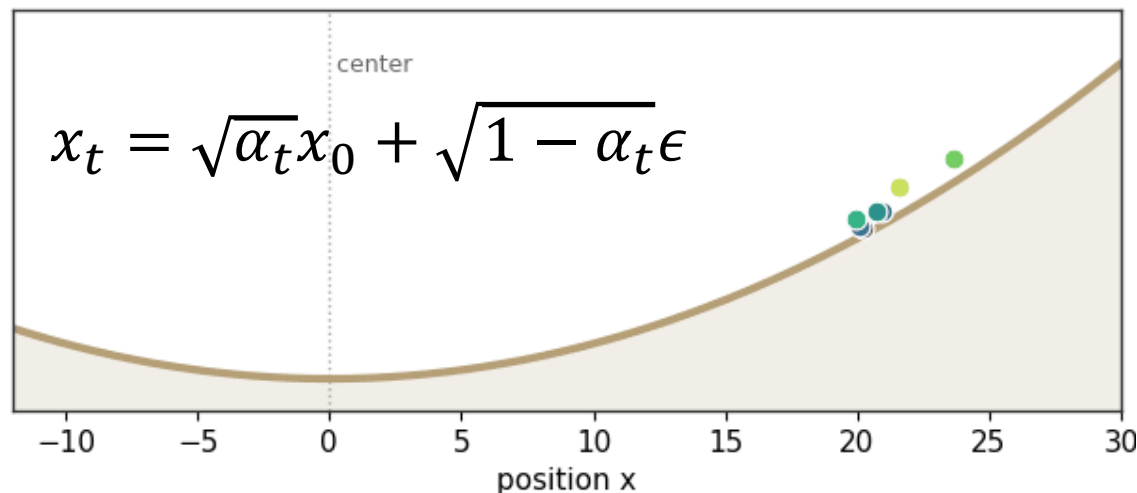
**Noise ( $\varepsilon$ )**

**Clean data ( $x_0$ )**

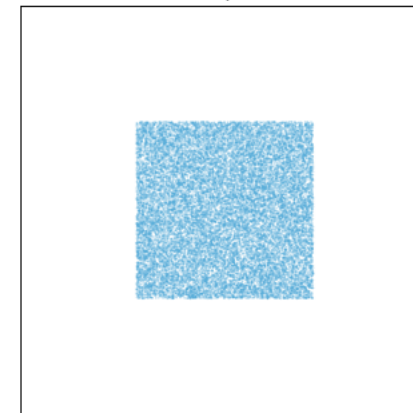
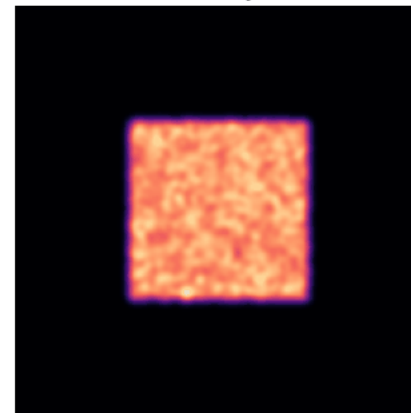
**Velocity / flow field ( $\frac{dx_t}{dt}$ ).**      E.g., FM:  $x_1 - x_0$

# Denoising Diffusion Probabilistic Model

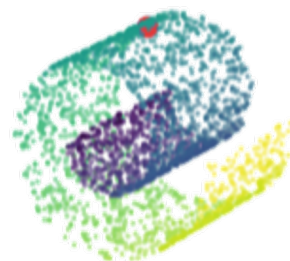
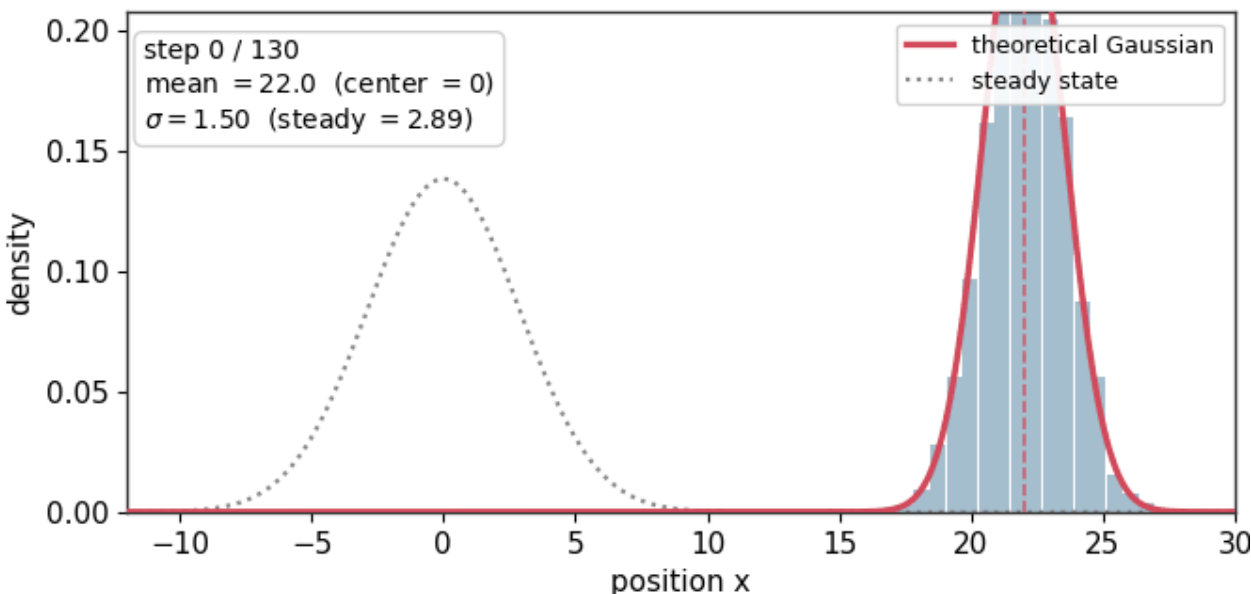
A few walkers in a bowl (quadratic potential: restoring pull to center)



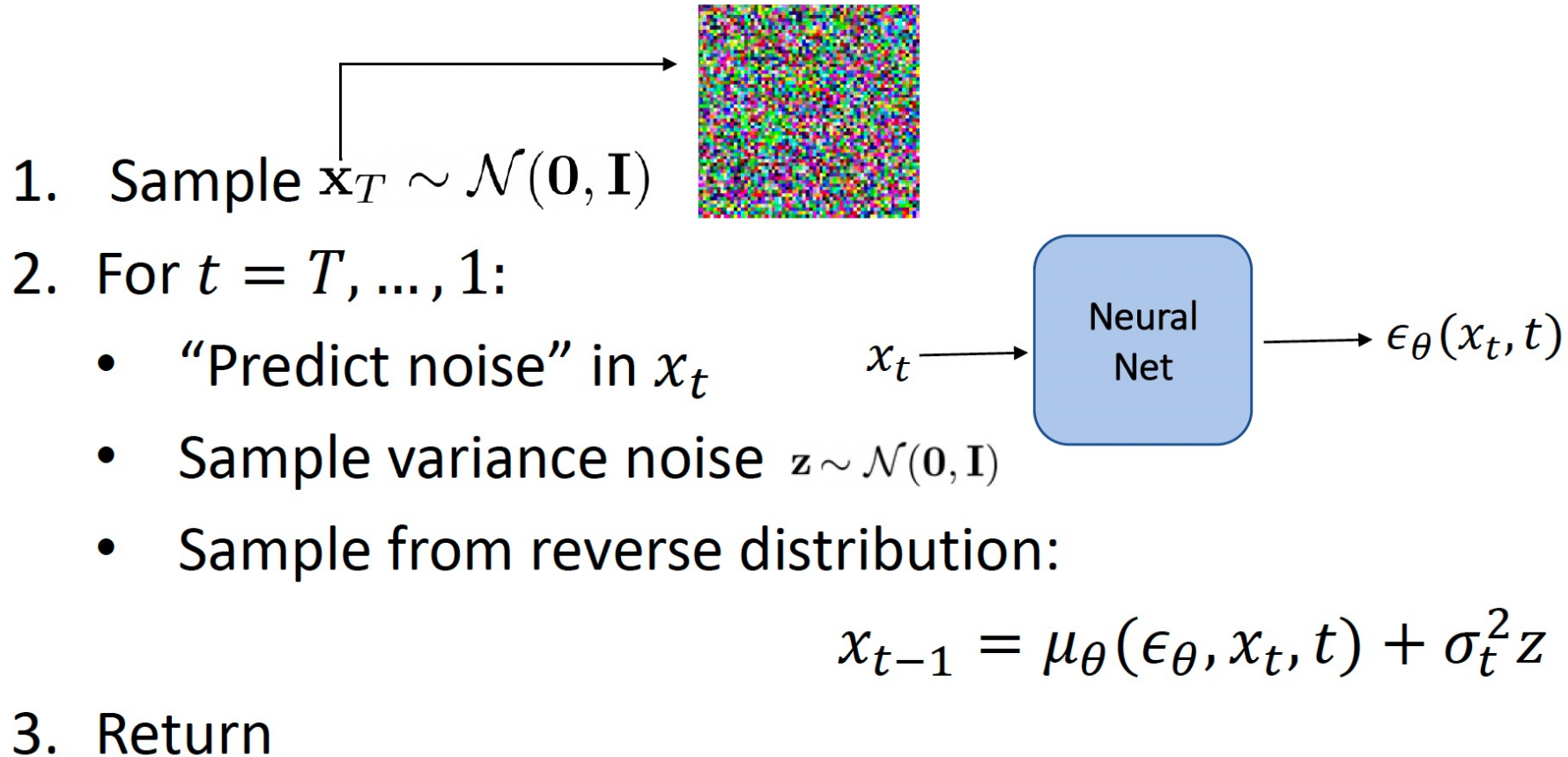
the data distribution: a filled square  
density samples



Many walkers relax to the steady-state Gaussian  $\mathcal{N}(0, D/k)$



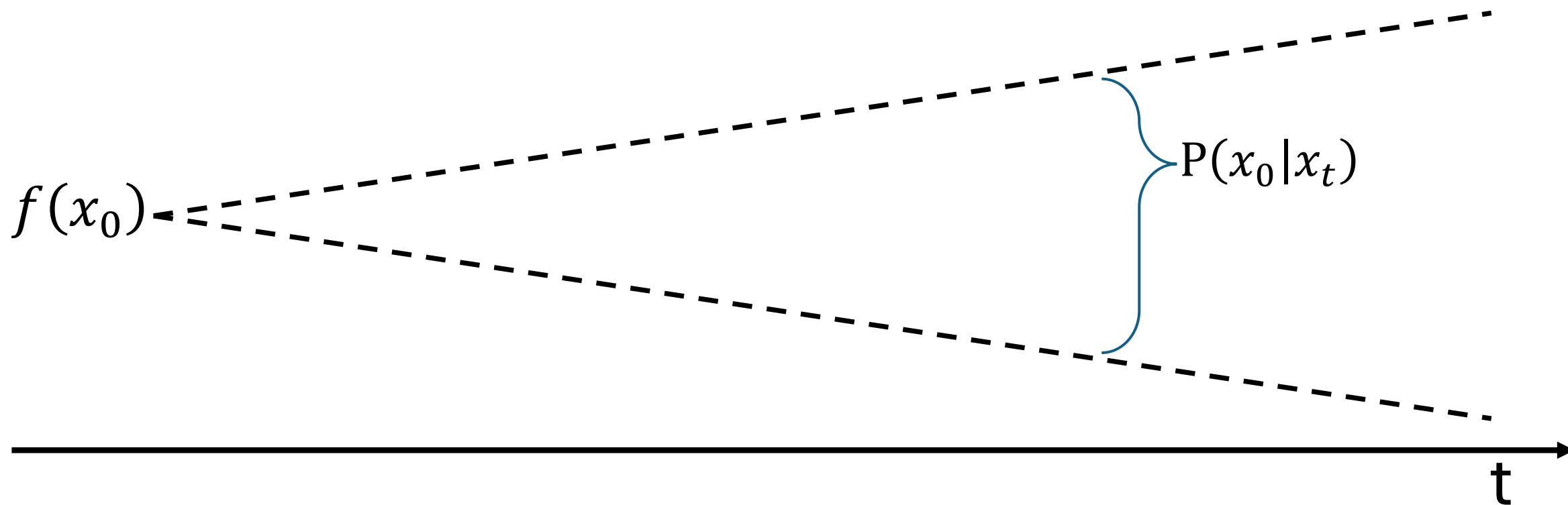
# Denoising Diffusion Probabilistic Model



# Denoising Diffusion Probabilistic Model

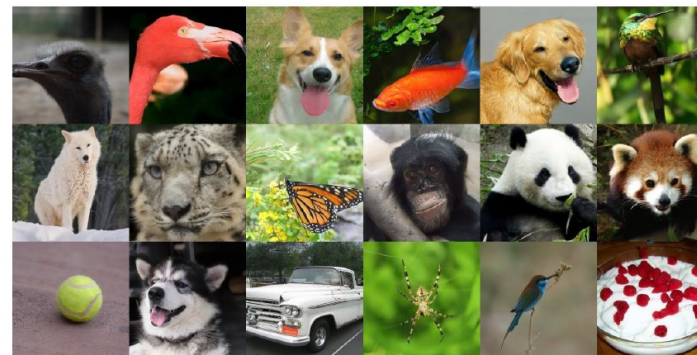
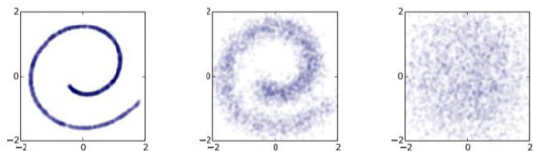


Training objective:  $f(x_t) \rightarrow x_0$



# The leading paradigm

## Papers



“Unsupervised Learning  
using Nonequilibrium  
Thermodynamics”

2015

“Denoising Diffusion  
Probabilistic Models  
(DDPM)”

2020



“Improved Denoising  
Diffusion  
Probabilistic Models”

2021

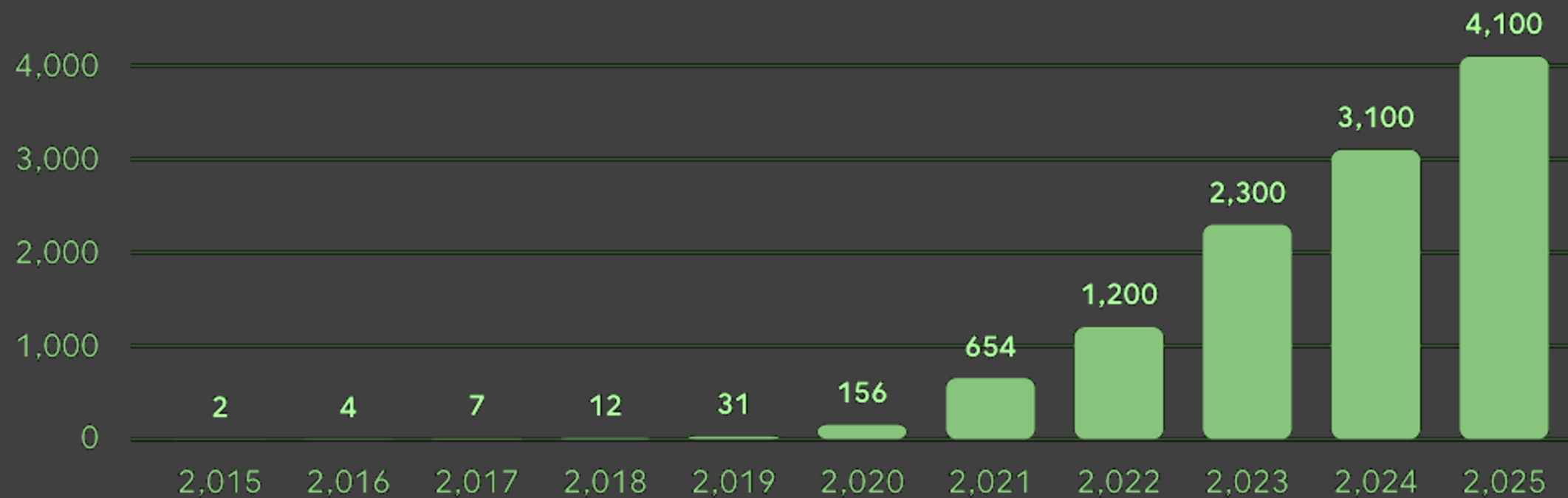


“Diffusion Models  
Beat GANs on Image  
Synthesis”

2021

# Sohl-Dickstein et al. 2015

Citations ↑



# GAN vs Diffusion





You

generate the picture depicting the joke: a snail on the back of a turtle saying "woohoo"



ChatGPT



Here's the image depicting the joke: a snail on the back of a turtle, exclaiming "woohoo!" The scene captures the snail's enthusiasm contrasted with the turtle's calm and steady pace. I hope you find it amusing! Let me know if there's anything else I can assist you with!



You

thats very good, but I want it to look like the snail feels the speed. maybe some wind on his skin or anything else that indicates that he feels he is going fast



ChatGPT



Here's the revised image where the snail truly feels the thrill of speed! The motion blur and the wind effects emphasize the snail's perceived rapid movement, even though the turtle remains at its usual slow pace. I hope this captures the humor and essence of the joke better. Let me know if there's anything else I can assist you with!

# Thank you!

Next week:

## Generative Models (2)

Truly understanding diffusion + special additional models

